

`jolto`: a Python library for interest-rate derivatives under rough volatility

Roberto Lucarini

November 12, 2026

Abstract

`jolto` is an open-source Python library for the calibration of, and valuation under, the Rough SABR Forward Market Model. It simulates the whole term structure of the forward rates jointly, requires no offline training and runs on a standard CPU. On the calibrated model, it prices Bermudan swaptions, CMS caplets and floorlets, and options on CMS spreads, with Greeks by adjoint algorithmic differentiation. This document sets out the theory and maps each method to the code that implements it. Code is available at github.com/robertolucarini/jolto.

Contents

1 Introduction **3**

2 Library Overview **3**

3 Interest-Rate Models **7**
3.1 Rough SABR Model 7
3.1.1 Conditional Monte Carlo 7
3.1.2 Closed-Form Approximation 8
3.2 Rough SABR Forward Market Model 9
3.2.1 Drift 9
3.2.2 Correlation 10
3.3 Swap Market Model Mapping 11

4 Simulation **12**
4.1 Discretisation Scheme 12
4.2 Quasi-Random Innovations 14
4.3 Engines 14
4.3.1 Rough SABR Engine 14
4.3.2 Rough SABR FMM Engine 16

5 Calibration **16**
5.1 Approximation and Model Management Optimization Framework 17
5.2 Stage 1: Rough Volatility Parameters 19
5.3 Stage 2: Correlation Matrix 20
5.3.1 Regularisation 20

6 Valuation **22**
6.1 Common Valuation Convention 22
6.2 Constant-Maturity Coupons 22
6.3 Constant-Maturity Spread Options 23
6.4 Bermudan Swaptions 23

7 Limitations **24**

A About the Regularisation of the Correlation Matrix **25**

1 Introduction

`jolto` is an open-source Python library for the calibration of, and valuation under, the Rough SABR Forward Market Model [1]. The library covers the whole chain from market quotes to the prices and Greeks of three exotic products. It calibrates the model by Monte Carlo guided by a closed-form surrogate, with no offline training and on a standard CPU. On the calibrated model, it prices Bermudan swaptions, CMS caplets and floorlets, and options on CMS spreads, with Greeks obtained by adjoint algorithmic differentiation [12].

The library addresses the cost of the calibration through four design choices. First, it follows the Approximation and Model Management Optimization framework (AMMO) [2]. Its trust-region subproblems run on the closed-form approximation of Fukasawa and Gatheral [9], corrected to first order at each iterate by a Monte Carlo pricer, which is called only to recalibrate the approximation. Second, the calibration pricer combines the conditional Monte Carlo estimator [22] with a control variate, and both engines draw their innovations from randomised Sobol sequences with antithetic variates [13][1]. Third, the engines are written as batched tensor operations in PyTorch [25], vectorised across paths and time steps. Automatic differentiation through these operations, which gives the Greeks, also gives the gradients of the trust-region subproblems and the Jacobians of the correlation calibration. At last, the calibration runs in two consecutive stages, first the rough volatility parameters on the one-year-tenor swaptions, then the correlation matrix.

The models and the numerical methods implemented in the library are not new. The dynamics are the Rough SABR model [9] and its extension to the forward curve [1]. The numerical methods includes the hybrid scheme [4], the conditional Monte Carlo [22] and the trust-region framework [2]. Our contribution is to bring them together in a single library for interest-rate derivatives. At the time of the writing, open-source code exists for the rough Bergomi model[2] and for the single-asset Rough SABR model[3], but we are not aware of a public implementation of the Rough SABR Forward Market Model.

2 Library Overview

`jolto` is a Python package built on PyTorch [25]. It run on the CPU by default and on a CUDA device when one is present. The package has five layers, each importing only from the layers below it.

`core/` holds the market-data loaders, the forward-curve bootstrap and the numerical primitives that every other component calls. These are the discrete Volterra kernel of the hybrid scheme and its variance compensator as per Section 4.1, the correlation matrix of Section 3.2.2, and the randomised Sobol, Brownian-bridge and antithetic construction of Section 4.2. The layer also holds the FMM engine of Section 4.3.2, which simulates the whole curve for valuation.

`calibration/` holds the one-dimensional pricers of the Rough SABR model, that is the Rough SABR engine of Section 4.3.1, the closed-form approximation of Section 3.1.2, the swap-market-model mapping of Section 3.3, and the two calibrators of Section 5. Both calibrators run the AMMO trust-region loop of Section 5.1, with the Rough SABR engine as the high-fidelity model and the closed form as the low-fidelity surrogate, and the second fits the correlation matrix

[1] The Bermudan pricer is the exception. Its simulation needs more dimensions than the Sobol generator admits, so it draws seeded pseudo-random innovations, still with antithetic variates (Section 6.4).

[2] The code at https://github.com/ryanmccrickerd/rough_bergomi simulates the rough Bergomi model of Bayer et al. [3] with the hybrid scheme of Bennedsen et al. [4] and prices with the estimators of McCrickerd and Pakkanen [22].

[3] The code at https://github.com/Datacarlo/AI_factory, in `src/model/equity/rough/rough_sabr`, simulates the Rough SABR model of Fukasawa and Gatheral [9] for an equity underlying on CUDA, with the hybrid scheme at $\kappa = 1$ and the kernel convolved by fast Fourier transform.

under Tikhonov regularisation. It also holds the two calibration pipelines that the command line runs, and the studies on the regularisation, `/regul/`, reported in Appendix A.

`valuation/` holds the pricers of Section 6, Bermudan swaptions by least-squares Monte Carlo [20], constant-maturity caplets and floorlets, and options on constant-maturity spreads. The two constant-maturity pricers carry self-consistency checks, which `jolto price` runs on request.

`api/` holds a build routine that assembles the calibrated model from the persisted calibration outputs, a trade specification for each product, and the pricing and simulation interfaces. The pricing interface returns the price, the pathwise Greeks obtained by adjoint algorithmic differentiation [12] and the timings of a run. The simulation interface returns the forward and discount curves along the simulated paths.

`cli/` holds the `jolto` command, whose subcommands run the pipeline of Figure 1. Table 1 lists the function that implements each method.

Table 1: Implementation of each method. Reference gives the equation or section that defines the method, Layer the subpackage of `jolto` that holds it, and Code the function, qualified by its module, or the method, qualified by its class.

Method	Reference	Layer	Code
Hybrid-scheme tail kernel	Eq. 30	`core`	`analytics.build_fractional_kernel`
Near-cell covariance	Eq. 32	`core`	`analytics.build_hybrid_cell_covariance`
Rough driver	Eq. 36	`core`	`analytics.build_hybrid_rough_driver`
Variance compensator	Eq. 37	`core`	`analytics.rough_vol_compensator`
RBM correlation matrix	Eq. 22	`core`	`analytics.build_rapisarda_correlation_matrix`
Sobol, bridge, antithetics	Sec. 4.2	`core`	`analytics.generate_sobol_bridge_increments`
FMM engine	Sec. 4.3.2	`core`	`RoughSABRFMM.simulate_forward_curve`
Rough SABR engine, Stage 1	Sec. 4.3.1	`calibration`	`RoughSABRCalibrator._make_hf_evaluator`
Rough SABR engine, Stage 2	Sec. 4.3.1	`calibration`	`pricers.mapped_smm_mc_pricer`
Closed-form approximation	Sec. 3.1.2	`calibration`	`RoughSABRCalibrator.rough_sabr_vol_ode`
SMM mapping weights	Eq. 25	`calibration`	`pricers.precompute_smm_weights`
Stage 1 calibration (AMMO)	Sec. 5.2	`calibration`	`RoughSABRCalibrator.calibrate`
Stage 2 calibration (AMMO)	Sec. 5.3	`calibration`	`CorrelationCalibrator.calibrate`
CMS caplet and floorlet	Sec. 6.2	`valuation`	`cms.cms_caplet_floorlet_price`
CMS spread option	Sec. 6.3	`valuation`	`cms.cms_spread_option_price`
Bermudan swaption (LSM)	Sec. 6.4	`valuation`	`bermudan_swaption.bermudan_pricer`
Model assembly		`api`	`assembly.build_calibrated_model`
Pricing interface		`api`	`get_price`
Curve simulation	Eq. 67	`api`	`simulate.simulate_curves`
Command line	Fig. 1	`cli`	`cli.main`

Figure 1 shows the pipeline from market data to pricing instruments with the calibrated model. Three subcommands of the `jolto` command run it in sequence, each writing its output to a file that the next one reads, so the model is calibrated once and priced as many times as required.

`jolto calibrate-sabr` reads the swaption volatility surface and fits the Rough SABR marginals of Section 5.2 to its one-year-tenor column. For each Hurst exponent on the candidate grid, it writes the volatility level $\alpha(T)$ and the spot-volatility correlation $\rho(T)$ per expiry, together with the global volatility of volatility ν. This stage fits in volatility space and needs no discount curve. `jolto calibrate-correlation` reads the discount curve, the at-the-money coterminal swaptions and the Stage 1 output, and calibrates the correlation matrix Σ of Section 5.3. `jolto price` reads the discount curve and both calibration outputs, rebuilds the model at the selected Hurst exponent, injects Σ through its Cholesky factor, and prices the chosen product. When Greeks are requested, automatic differentiation tracks the initial forward rates $F_j(0)$ and the volatility levels α_j through the simulation, and one adjoint pass at the end returns a PV01 and a vega along the tenor structure. A fourth subcommand, `jolto simulate-curves`, runs the same simulation without a product and writes the forward rates and the discount bonds of Equation 67 at chosen dates.

Path counts, time steps, the Hurst grid, the trust-region constants and the rule that selects the regularisation weight are set in two configuration modules, one for calibration and one for valuation. Each subcommand checks the entries it needs before it runs. Every simulation draws from fixed seeds, so repeated runs use identical paths.

The validation scripts in `tests/` read the same files, so the bias and convergence tests run against the model that is priced, and `tests/oos.py` reprices swaptions left out of the calibration, with confidence intervals obtained by resampling whole smiles.

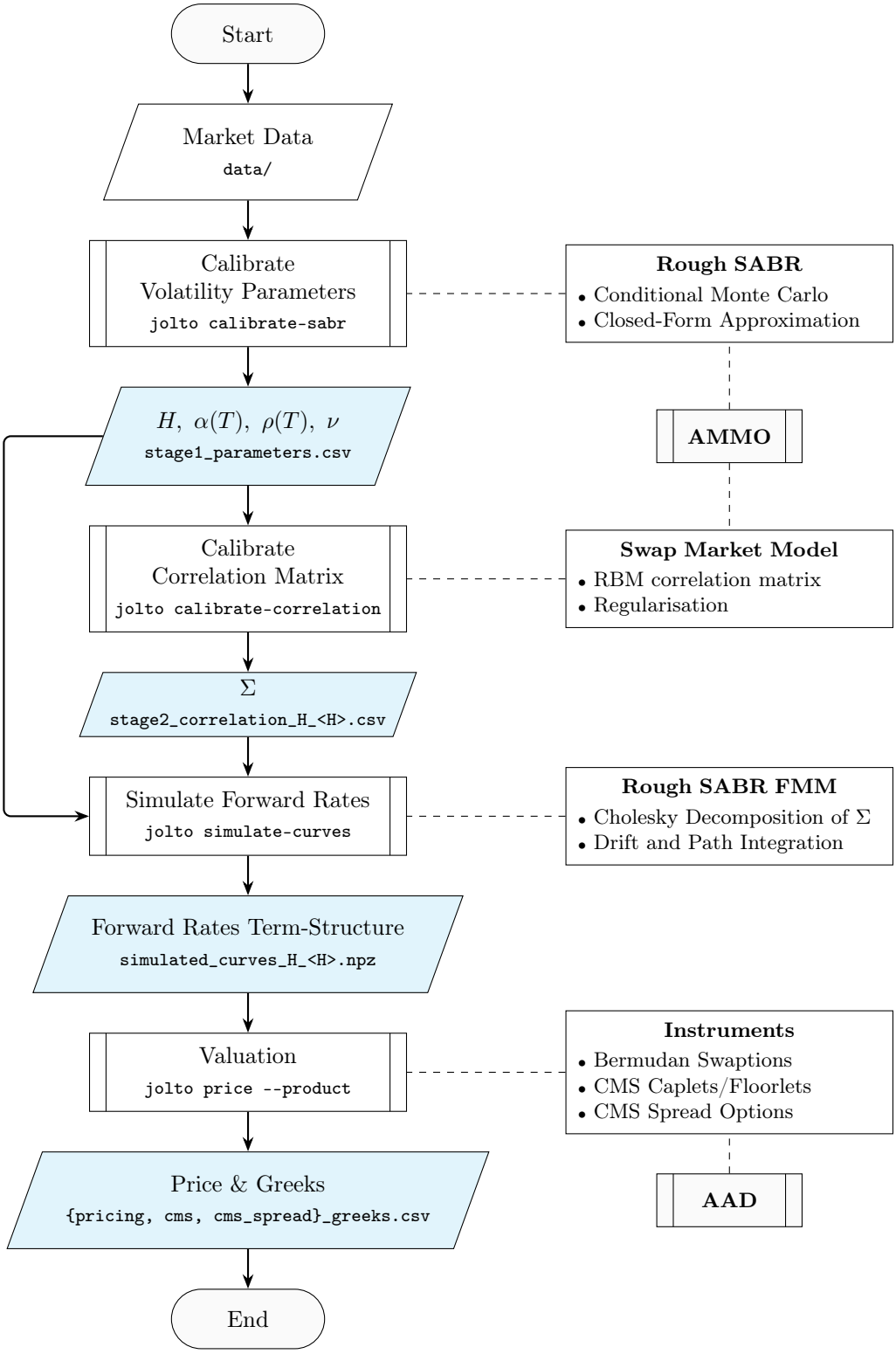


Figure 1: The `jolto` pipeline. Rectangles with side bars are the four processing stages, parallelograms their inputs and outputs, and the boxes on the right, joined by dashed lines, name the component of the library that each stage runs. Market data are the discount curve and the swaption volatility surface. Each stage names the subcommand that runs it, the inputs are read from `data/`, and each output names the file written to `results/`. `jolto simulate-curves` and `jolto price` both run the simulation stage. `jolto simulate-curves` stops after it and writes the simulated curves, while `jolto price` passes them to the valuation in memory. The two calibration stages share the AMMO loop of Section 5.1; the Stage 1 output feeds both the correlation calibration and the simulation; the Greeks are produced by adjoint algorithmic differentiation (AAD).

3 Interest-Rate Models

3.1 Rough SABR Model

Let $F(t)$ be the forward swap rate underlying a swaption, and let $\mathbb{Q}^A$ be the annuity measure of its swap, under which F is a martingale. Following Fukasawa and Gatheral [9], under $\mathbb{Q}^A$, F evolves as

$$dF(t) = \alpha(t)C(F(t))dW(t), \tag{1}$$

$$\alpha(t) = \alpha(0)V(t), \tag{2}$$

$$V(t) = \exp\left(\nu\tilde{Z}(t) - \mathcal{V}(t)\right), \tag{3}$$

where $C(\cdot)$ is the local volatility function, set to $C(F(t)) = 1$ for the normal specification used throughout. The total volatility $\alpha(t)$ is decomposed into $\alpha(0)$, the initial deterministic scalar anchoring the level of the at-the-money implied volatility, and $V(t)$, a unit-mean lognormal stochastic driver. The parameter ν is the constant volatility of volatility, and $W(t)$ is a standard Brownian motion.

The fractional driver $\tilde{Z}(t)$ is a Riemann–Liouville fractional integral of a second standard Brownian motion $Z(t)$, correlated with the rate driver through $dW(t)dZ(t) = \rho dt$, and is defined through the power-law Volterra kernel $K(t, s)$,

$$\tilde{Z}(t) = \int_0^t K(t,s)dZ(s), \quad \text{where} \quad K(t,s) = \frac{(t-s)^{H-1/2}}{\Gamma(H+1/2)}. \tag{4}$$

with $H \in (0, 1/2]$. At $H = 1/2$ the kernel equals one, $\tilde{Z} = Z$, and the model reduces to the classical SABR. For $H < 1/2$ the exponent $H - 1/2$ is negative, and $K(t, s)$ diverges as s approaches t[4].

For the underlying forward rate to retain its martingale property, the exponential volatility process must satisfy $\mathbb{E}^A[V(t)] = 1$. However, discretising a fractional process creates a mismatch between its theoretical continuous variance and the variance generated on the computational grid. This discrepancy introduces an artificial drift into the exponentiated paths and breaks the unit-mean condition. To resolve this, a deterministic variance compensator $\mathcal{V}(t_i)$ is introduced. In the continuous limit, it is the L^2 norm of the Volterra kernel, integrated over each discrete time bin and summed across the path history,

$$\mathcal{V}(t_i) = \frac{\nu^2}{2}\sum_{j=1}^{i}\int_{t_{j-1}}^{t_j}\left(\frac{(t_i-s)^{H-1/2}}{\Gamma(H+1/2)}\right)^2 ds. \tag{5}$$

3.1.1 Conditional Monte Carlo

In the conditional Monte Carlo [22, 29], the rate driver of Equation 1 is decomposed into a component parallel to the volatility driver and one orthogonal to it,

$$dW(t) = \rho\, dZ(t) + \sqrt{1-\rho^2}\, dZ^{\perp}(t), \tag{6}$$

where $Z^{\perp}$ is a standard Brownian motion independent of Z. Substituting this decomposition into the forward rate dynamics of Equation 1 gives

$$dF(t) = \alpha(0)V(t)\left(\rho\, dZ(t) + \sqrt{1-\rho^2}\, dZ^{\perp}(t)\right), \tag{7}$$

[4]It remains square-integrable, since $\int_0^t (t-s)^{2H-1}\, ds = t^{2H}/(2H)$ is finite for $H > 0$, so $\tilde{Z}(t)$ is well defined.

and, at the expiry T of the swaption,

$$F(T) = F(0) + \alpha(0)\rho \int_0^T V(t)\, dZ(t) + \alpha(0)\sqrt{1-\rho^2} \int_0^T V(t)\, dZ^{\perp}(t). \tag{8}$$

Let $\mathcal{F}_T^Z = \sigma\{Z(s) : 0 \le s \le T\}$ be the σ-algebra generated by the volatility driver up to the terminal horizon. Conditional on $\mathcal{F}_T^Z$, the whole path of $V(t)$ is known, since it depends on Z alone, and in Equation 8 the integral against $dZ(t)$ is $\mathcal{F}_T^Z$-measurable, so it is known as well and acts as a constant. Since $Z^{\perp}$ is independent of $\mathcal{F}_T^Z$, it remains a standard Brownian motion conditional on $\mathcal{F}_T^Z$, while $V(t)$ is a fixed function of time. The integral against $dZ^{\perp}(t)$ is then a Wiener integral of a deterministic integrand, and hence centred Gaussian. The terminal forward rate $F(T)$, a constant plus a centred Gaussian, is therefore conditionally normal.

Its conditional mean collects the constant terms,

$$F_{cond} := \mathbb{E}^A[F(T)|\mathcal{F}_T^Z] = F(0) + \alpha(0)\rho \int_0^T V(t) dZ(t), \tag{9}$$

and its conditional variance follows from Itô's isometry applied to the orthogonal integral,

$$\sigma_{cond}^2 := \mathrm{Var}(F(T)|\mathcal{F}_T^Z) = \alpha(0)^2(1-\rho^2) \int_0^T V(t)^2 dt. \tag{10}$$

Since the conditional distribution is normal, evaluating the expected payoff of a European swaption under this measure reduces to closed form. Instead of simulating the full terminal payoff, we evaluate the Bachelier formula along the simulated paths using the conditional moments F_{cond} and σ_{cond}^2, as shown in Section 4.3.1.

3.1.2 Closed-Form Approximation

Fukasawa and Gatheral [9] derive an asymptotic approximation of the implied volatility under the Rough SABR model. In the Bachelier case, it is the product of a level and the shape factor $|y|/\sqrt{G(y)}$, where y is a scaled strike offset and G solves an ordinary differential equation[5]. The equation has a closed-form only at $H = 1/2$, where it gives the classical SABR solution

$$G_{1/2}(y) = 4\left(\log \frac{\sqrt{1+\rho y + y^2/4} - \rho - y/2}{1-\rho}\right)^2, \tag{11}$$

and at $H = 0$, where it gives the rough limit

$$G_0(y) = \log(1 + 2\rho y + y^2) + \frac{2\rho}{\sqrt{1-\rho^2}}\left(\arctan\frac{\rho}{\sqrt{1-\rho^2}} - \arctan\frac{y+\rho}{\sqrt{1-\rho^2}}\right). \tag{12}$$

For general H the equation must be solved numerically.

The authors interpolate between the two solutions and obtain the closed form

$$G_A(y) = (2H+1)^2 \left\{ \frac{3(1-2H)}{2H+3}\, G_0\!\left(\frac{y}{2H+1}\right) + \frac{2H}{2H+3}\, G_{1/2}\!\left(\frac{2y}{2H+1}\right)\right\}, \tag{13}$$

which keeps the exact at-the-money skew for every H.

The closed form used by the library is the normal SABR formula of Hagan et al. [15] with its shape factor replaced by $|y|/\sqrt{G_A(y)}$. The level is $\alpha(0)$ with Hagan's correction term, in

[5] In the original work, the level U is the square root of the mean forward variance $\xi_0(s) = \mathbb{E}^A[\alpha(s)^2]$ over $[0,T]$, and their $Y = \kappa(T)\, k/U$, where $\kappa = 2\nu K$ is the kernel of ξ, since squaring α doubles the exponent of Equation 3. The library sets $U = \alpha(0)$ in both (Equation 15).

which $\nu^2 T$ is replaced by the variance of the rough driver[6], and y is the scaled strike offset of Fukasawa and Gatheral [9],

$$\sigma_{proxy}(k,T) = \alpha(0)\left(1 + \frac{2-3\rho^2}{24}\,\nu^2 \operatorname{Var}\tilde{Z}(T)\right)\frac{|y|}{\sqrt{G_A(y)}}, \tag{14}$$

with

$$y = \frac{2\nu}{\Gamma(H+1/2)}\,T^{H-1/2}\,\frac{k}{\alpha(0)}, \tag{15}$$

where $\operatorname{Var}\tilde{Z}(T) = T^{2H}/(2H\,\Gamma(H+1/2)^2)$. At $H = 1/2$, where $G_A = G_{1/2}$ and $\operatorname{Var}\tilde{Z}(T) = T$, the closed form reduces to the normal SABR formula.

3.2 Rough SABR Forward Market Model

Let $\mathcal{T} = \{T_0 < T_1 < \cdots < T_N\}$ be a tenor structure with year fractions $\tau_j = T_j - T_{j-1}$, let $P(t,T)$ be the price at time t of the zero-coupon bond that pays one unit of currency at T, with $P(T,T) = 1$, and let $F_j(t)$ be the backward-looking forward term rate for the accrual period $(T_{j-1}, T_j]$, $j = 1, \ldots, N$.

In the Rough SABR Forward Market Model of Adachi et al. [1], the forward rates evolve under the terminal measure $\mathbb{Q}^{T_N}$, associated with the numéraire $P(t,T_N)$, as

$$dF_j(t) = \mu_j(t)\,dt + \gamma_j(t)\,\alpha_j(0)V_j(t)\,C(F_j(t))\,dW_j(t), \tag{16}$$

$$V_j(t) = \exp\left(\nu\tilde{Z}(t) - \mathcal{V}_j(t)\right), \tag{17}$$

where μ_j is the state-dependent no-arbitrage drift, $\alpha_j(0)$ the level of the at-the-money volatility of F_j, $C \equiv 1$ for normal volatilities, $\tilde{Z}$ the fractional driver of Equation 4, and $\mathcal{V}_j$ the compensator of Equation 5. The standard Brownian motions W_j and Z are correlated through $dW_j(t)\,dZ(t) = \rho_j\,dt$. The deterministic function γ_j equals 1 for $t \le T_{j-1}$ and 0 thereafter, so that F_j is frozen once it sets [7].

3.2.1 Drift

The drift follows from the derivation of Lyashenko and Mercurio [21]. Under the terminal measure $\mathbb{Q}^{T_N}$ and for $t \le T_{j-1}$, where $\gamma_k = 1$ for every $k \ge j$, the drift of the forward rate F_j at time t is

$$\mu_j(t) = -\alpha_j(0)V_j(t)\sum_{k=j+1}^{N}\Sigma_{j,k}\frac{\tau_k\alpha_k(0)V_k(t)}{1+\tau_k F_k(t)}, \tag{18}$$

where $\Sigma_{j,k}$ is the instantaneous correlation of W_j and W_k, $dW_j(t)\,dW_k(t) = \Sigma_{j,k}\,dt$. For $t > T_{j-1}$ the drift is zero, since $\gamma_j = 0$, and so it is for $j = N$ at all times, since the sum is empty and $\mathbb{Q}^{T_N}$ is the forward measure of F_N.

All the $V_j(t)$ share the driver $\tilde{Z}(t)$, the compensator of Equation 5 and the single global ν, so they coincide, and the product $V_j(t)V_k(t)$ equals $V_j(t)^2$ and factors out of the sum. The drift then depends on the rates only through the denominators $1 + \tau_k F_k(t)$. These involve the simulated values of all longer-dated forwards, so a scheme that keeps the drift exact must advance the curve one time step at a time. In turn, we keep the forward rates in the drift fixed at their time-zero values.

[6] The replacement is a heuristic. It recovers Hagan's term at $H = 1/2$ but is not derived for $H < 1/2$, which suffices because AMMO corrects the closed form against the Rough SABR engine at every iterate (Section 5.1).

[7] This is a simplification of the original specification of Lyashenko and Mercurio [21], in which γ_j decays linearly over the accrual period $(T_{j-1}, T_j]$ as the backward-looking rate accrues, and it is justified by the annual accrual of the forward grid adopted here. The library does not apply γ_j and evolves each forward past its fixing date, which leaves every price unchanged, since no pricer reads a forward after it sets.

"Freezing" the drift is a standard approximation in the LIBOR market model [5, 28]. We freeze the denominators only and keep the volatility factor in full, since it does not depend on the rates and is already simulated on every path. The drift becomes

$$\mu_j(t) \approx \mu_j(0)\, V_j(t)^2, \tag{19}$$

with

$$\mu_j(0) = -\alpha_j(0) \sum_{k=j+1}^{N} \Sigma_{j,k} \frac{\tau_k \alpha_k(0)}{1 + \tau_k F_k(0)} \tag{20}$$

the drift of Equation 18 at $t = 0$, where $V_j(0) = 1$. Since $\mu_j(0)$ is deterministic, no forward needs the simulated curve of the previous step, and each integrates in the single cumulative sum of Equation 48.

The approximation is exact on a single forward, which has no inter-forward drift, and `tests/test_drift_freezing_bias.py` measures the error otherwise.

3.2.2 Correlation

Let Σ be the $(N+1) \times (N+1)$ instantaneous correlation matrix of the Brownian motions $Z, W_1, \ldots, W_N$ that drive the volatility and the N forward rates. To keep Σ positive semi-definite with unit diagonal for every value of the calibrated parameters, we adopt the hyperspherical parameterisation of Rapisarda et al. [27],

$$\Sigma = BB^\top, \tag{21}$$

where B is a lower-triangular pseudo square root matrix with entries $b_{i,j}$, $i, j = 0, \ldots, N$, given in terms of angles $\omega_{i,j} \in [0, \pi]$ by

$$b_{i,j} = \begin{cases} \cos\omega_{i,j} \prod_{k=0}^{j-1} \sin\omega_{i,k}, & 0 \le j < i \\ \prod_{k=0}^{j-1} \sin\omega_{i,k}, & j = i \\ 0, & j > i, \end{cases} \tag{22}$$

with the empty product for $j = 0$ set equal to one, so that the first column of B reads $b_{0,0} = 1$ and $b_{i,0} = \cos\omega_{i,0}$ for $i > 0$.

Generalising to the full system, matrix B becomes

$$B = \begin{pmatrix} 1 & 0 & 0 & \ldots & 0 \\ \cos\omega_{1,0} & \sin\omega_{1,0} & 0 & \ldots & 0 \\ \cos\omega_{2,0} & \cos\omega_{2,1}\sin\omega_{2,0} & \sin\omega_{2,0}\sin\omega_{2,1} & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \cos\omega_{N,0} & \cos\omega_{N,1}\sin\omega_{N,0} & \cos\omega_{N,2}\sin\omega_{N,0}\sin\omega_{N,1} & \ldots & \prod_{k=0}^{N-1}\sin\omega_{N,k} \end{pmatrix}. \tag{23}$$

Following Adachi et al. [1], the volatility driver $Z(t)$ is placed at index 0 of the system. Since B is lower-triangular, its first row is $(1, 0, \ldots, 0)$, so $Z(t)$ coincides with the first orthogonal shock and the spot-volatility correlation between the i-th forward rate and $Z(t)$ reduces to $\Sigma_{i,0} = b_{i,0} = \cos\omega_{i,0} = \rho_i$, isolated in the first column of the matrix. Here ρ_i is the correlation $dW_i(t)\, dZ(t) = \rho_i\, dt$ of Section 3.2, the parameter calibrated in the first stage that controls the skew of the i-th marginal smile.

The library builds B by element-wise operations on the lower-triangular matrix Ω of the angles $\omega_{i,j}$, without loops over its entries. Let $\mathcal{C} = \cos(\Omega)$ and $\mathcal{S} = \sin(\Omega)$ denote the trigonometric matrices, taken element-wise, and let $\mathcal{P}$ be the cumulative product matrix defined by $\mathcal{P}_{i,j} = \prod_{k=0}^{j-1} \mathcal{S}_{i,k}$. The library obtains every entry of $\mathcal{P}$ at once. It shifts each row of $\mathcal{S}$ one column to the right, fills column 0 with ones, the value of the empty product, and takes the

cumulative product along the row, so that row i of $\mathcal{P}$ reads $(1,\ \mathcal{S}_{i,0},\ \mathcal{S}_{i,0}\mathcal{S}_{i,1},\ \dots)$. Setting the diagonal of $\mathcal{C}$ to one, the library forms B as the lower triangle of the element-wise product $\mathcal{C}\odot\mathcal{P}$, so that $b_{i,j}=\mathcal{C}_{i,j}\mathcal{P}_{i,j}$ for $j<i$ and $b_{i,i}=\mathcal{P}_{i,i}$.[8]

Finally, the free elements of Ω are calibrated to fit the observed market-implied volatilities, with the final correlation matrix computed as $\Sigma=BB^{\top}$.

3.3 Swap Market Model Mapping

To calibrates the correlation matrix, the model must price swaptions at every iteration. A swaption depends on all the forward rates of the underlying swap, and the joint distribution of these rates is not known in closed form, so each price requires simulating them together up to the expiry. Adachi et al. [1] propose instead an approximate model for the swap rate alone, a rough Bergomi forward swap model. They prove that its implied volatilities agree with those of the full model to leading order as the expiry tends to zero, for strikes near the money.

Let $S_{m,n}(t)$ be the forward swap rate at time t of an interest-rate swap with first reset date T_m and payment dates $T_{m+1},\dots,T_n$, $m<n$. In terms of the zero-coupon bonds P, it reads

$$S_{m,n}(t)=\frac{P(t,T_m)-P(t,T_n)}{A_{m,n}(t)},\quad \text{where}\quad A_{m,n}(t)=\sum_{k=m+1}^{n}\tau_k P(t,T_k). \tag{24}$$

Because the forward swap rate is a linear combination of the underlying forward rates $F_k(t)$ with state-dependent weights, its instantaneous volatility can be approximated by freezing these weights at their initial values [28]. Hull and White [17] freeze instead the derivatives of the swap rate with respect to the forward rates, and their closed form is derived in [1].

Let Π_k denote the sensitivity of the swap rate to the single forward rate F_k

$$\Pi_k=\frac{\partial S_{m,n}(0)}{\partial F_k(0)}=\frac{\tau_k P(0,T_k)}{A_{m,n}(0)P(0,T_{k-1})}\left(P(0,T_n)+S_{m,n}(0)\sum_{j=k}^{n}\tau_j P(0,T_j)\right) \tag{25}$$

with $k=m+1,\dots,n$. To project the correlation down to a single scalar variance for the swap rate, the volatility-scaled weights are defined as

$$w_k=\Pi_k\alpha_k, \tag{26}$$

where α_k is the base normal volatility of the k-th forward rate. The mapped effective normal volatility α_{SMM} of the swaption is then the square root of the quadratic form of the $N\times N$ forward-rate submatrix Σ_F, extracted from the $(N+1)\times(N+1)$ system, excluding the common rough factor,

$$\alpha_{\mathrm{SMM}}=\sqrt{w^{\top}\Sigma_F w}. \tag{27}$$

Similarly, the effective spot-volatility correlation ρ_{SMM} for the swap rate process is defined as a variance-weighted linear combination of the marginal forward rate correlations ρ_k,

$$\rho_{\mathrm{SMM}}=\frac{1}{\alpha_{\mathrm{SMM}}}\sum_{k=m+1}^{n}\rho_k w_k. \tag{28}$$

The volatility of volatility is a single global parameter, and the mapped process inherits it unchanged, $\nu_{\mathrm{SMM}}=\nu$.

[8] The product gives the three cases of Equation 22 entry by entry. For $j<i$, $\mathcal{C}_{i,j}\mathcal{P}_{i,j}=\cos\omega_{i,j}\prod_{k=0}^{j-1}\sin\omega_{i,k}=b_{i,j}$. For $j=i$, $b_{i,i}$ has no cosine factor, so setting $\mathcal{C}_{i,i}=1$ leaves $\mathcal{P}_{i,i}=b_{i,i}$. For $j>i$, $b_{i,j}=0$, and taking the lower triangle sets these entries to zero. In the library Ω is zero on and above its diagonal, so $\cos\omega_{i,i}=\cos 0=1$ already, and every $\mathcal{P}_{i,j}$ with $j>i$ contains the factor $\sin\omega_{i,i}=\sin 0=0$ and vanishes.

Since all the forward rates share the volatility process V, freezing the weights reduces the swap rate to a single process. Under the annuity measure $\mathbb{Q}^{m,n}$, $S_{m,n}$ then follows the Rough SABR model of Section 3.1, with volatility level α_{SMM}, correlation ρ_{SMM}, and the ν and H of Stage 1. As a result, Stage 2 can price each swaption with the Rough SABR engine of Section 4.3.1, without simulating the forward curve.

4 Simulation

The calibration runs on the Rough SABR engine, which simulates the single rate of Section 3.1, and the valuation on the FMM engine, which simulates the whole forward curve of Section 3.2. Both use the same discrete Volterra kernel and variance compensator.

4.1 Discretisation Scheme

The rough volatility process requires integrating the continuous kernel $K(t,s)$ with respect to a standard Brownian motion. Because the Hurst exponent is restricted to the rough regime ($H < 1/2$), the exponent $H - 1/2$ is strictly negative, so the kernel diverges as the integration variable s approaches the observation time t. If this process is simulated using a standard forward Riemann sum, evaluating the kernel at the grid points underestimates the contribution of the cells nearest the singularity to the variance of the driver.

To remove this bias, we implement the hybrid scheme introduced by Bennedsen et al. [4], which splits the stochastic integral into two parts by the time lag $k = i - j + 1$. For the first κ intervals adjacent to the singularity ($k \leq \kappa$), the scheme avoids discrete point evaluation. Instead of reducing these lags to a deterministic weight, it draws their stochastic integrals exactly from their joint Gaussian law, conditionally on the Brownian increment that also drives the rest of the path. For the remaining historical tail ($k > \kappa$), where the continuous kernel is smooth, the stochastic integral is approximated by a Riemann sum populating a strictly lower-triangular kernel matrix.

In this work, we set $\kappa = 1$. The case $\kappa = 0$ recovers the pure Volterra-Euler scheme, in which every lag is approximated by the cell-average weight of Equation 30.

Let t_i be an observation time and $(t_{j-1}, t_j]$, with $j \leq i$, a cell of the grid, and let $k = i - j + 1 \geq 1$ be the lag of the cell from the observation.

For a tail cell, the kernel is replaced by a single value on the cell. Writing $u = t_i - s \in ((k-1)\Delta t, k\Delta t]$ for the argument of the kernel, the value is taken at

$$u = b_k^* \Delta t, \qquad \text{where} \qquad b_k^* = \left(\frac{k^{H+1/2} - (k-1)^{H+1/2}}{H + 1/2} \right)^{\frac{1}{H-1/2}} \tag{29}$$

is the point at which the kernel equals its average over the cell, and also the choice that minimises the L^2 distance between the discrete and the continuous kernel [4]. The tail weight is the kernel evaluated at this point,

$$h_k = \frac{(b_k^* \Delta t)^{H-1/2}}{\Gamma(H + 1/2)} = \frac{\Delta t^{H-1/2}}{\Gamma(H + 1/2)} \frac{k^{H+1/2} - (k-1)^{H+1/2}}{(H + 1/2)}, \tag{30}$$

so that $h_k \Delta t$ is the exact integral of the kernel over the cell. Then, the weights are collected in the lower-triangular matrix K^{tail}, which carries h_{i-j+1} in position (i,j) when $i - j + 1 > \kappa$ and zero otherwise, so that the tail contribution is the matrix product $K^{\text{tail}} \Delta Z$.

For a near cell, the contribution is the power-weighted Wiener integral

$$\xi_{j,k} = \int_{t_{j-1}}^{t_j} [(j - 1 + k)\Delta t - s]^{H-1/2} dZ(s), \tag{31}$$

divided by $\Gamma(H+1/2)$. Within cell j, the increment and the near integrals form the centred Gaussian vector $(\Delta Z_j, \xi_{j,1}, \ldots, \xi_{j,\kappa}) \sim \mathcal{N}(0, C^{\text{near}})$, whose covariance depends only on H and Δt; for $\kappa = 1$,

$$C^{\text{near}} = \begin{pmatrix} \Delta t & \dfrac{\Delta t^{H+1/2}}{H+1/2} \\ \dfrac{\Delta t^{H+1/2}}{H+1/2} & \dfrac{\Delta t^{2H}}{2H} \end{pmatrix}, \tag{32}$$

with rows and columns indexed from 0, index 0 referring to ΔZ_j. Since ΔZ_j already drives the tail, $\xi_{j,1}$ is not drawn independently but conditionally on it, as

$$\xi_{j,1} = b_{\text{near}}\, \Delta Z_j + s_{\text{near}}\, \varepsilon_j, \qquad \varepsilon_j \sim \mathcal{N}(0,1) \text{ independent of } \Delta Z_j, \tag{33}$$

where b_{near} is the regression coefficient of $\xi_{j,1}$ on ΔZ_j and s^2_{near} the residual variance[9],

$$b_{\text{near}} = \frac{C^{\text{near}}_{01}}{C^{\text{near}}_{00}} = \frac{\Delta t^{H-1/2}}{H+1/2}, \tag{34}$$

$$s^2_{\text{near}} = C^{\text{near}}_{11} - \frac{(C^{\text{near}}_{01})^2}{C^{\text{near}}_{00}} = \frac{\Delta t^{2H}(H-1/2)^2}{2H(H+1/2)^2}. \tag{35}$$

The driver at t_i is the sum of the two contributions. The lag-k integral of cell j enters the driver at t_i with $i = j-1+k$, so for $\kappa = 1$ the near term at t_i is the lag-1 integral of cell i itself,

$$\tilde{Z}(t_i) = \sum_j K^{\text{tail}}_{i,j}\, \Delta Z_j + \frac{1}{\Gamma(H+1/2)} \sum_{k=1}^{\min(i,\kappa)} \xi_{i-k+1,\,k}. \tag{36}$$

By simulating the singular cells exactly, the scheme removes the leading near-diagonal bias of the Volterra-Euler discretisation at small H, while the tail retains the cost of a standard Euler convolution.

The continuous volatility process has unit mean by construction, and the discrete one must be made to share this property. Since $\tilde{Z}(t_i)$ is Gaussian, $\mathbb{E}[V(t_i)] = 1$ holds if and only if the compensator equals half its variance, $\mathcal{V}(t_i) = \frac{\nu^2}{2} \operatorname{Var} \tilde{Z}(t_i)$. Each cell enters Equation 36 either through its increment or through one of its near integrals, and never both. The terms are therefore independent across cells, and the variance is the sum of the near and tail variances,

$$\mathcal{V}(t_i) = \frac{\nu^2}{2} \left[\frac{1}{\Gamma(H+1/2)^2} \sum_{k=1}^{\min(i,\kappa)} C^{\text{near}}_{kk} + \Delta t \sum_j (K^{\text{tail}}_{i,j})^2 \right], \tag{37}$$

where $\Delta t = \operatorname{Var}(\Delta Z_j)$ and $C^{\text{near}}_{kk} = \operatorname{Var}(\xi_{j,k})$.

In a tail cell the scheme keeps only the regression part of the near-cell construction. The cell-average weight h_k equals the regression coefficient of the exact cell integral on its increment ΔZ_j, and the residual that the near cell draws through $s_{\text{near}}\, \varepsilon_j$ is dropped. As a result, the discrete driver has a smaller variance than the exact one[10]. The compensator of Equation 37 tracks the discrete variance, and $\mathbb{E}[V(t_i)] = 1$ holds on every grid, although the dynamics of V differ from the continuous ones. The error is measured with `tests/test_euler_weak_error.py`.

Finally, the driver $\tilde{Z}$ and the bracketed variance of Equation 37 depend on H, the grid, and the increments but not on ν, α or ρ, so the calibration builds them once per Hurst exponent and reuses them across optimiser iterations.

[9] The library implements the factored form. As $H \to 1/2$ the two terms of the difference both tend to Δt, and subtracting two nearly equal numbers loses precision in floating point, whereas the factor $(H-1/2)^2$ produces the small result directly.

[10] Gassiat [10] shows that the deficit is of order Δt^{2H}.

4.2 Quasi-Random Innovations

To reduce the variance of the Monte Carlo estimator, the noise innovations are generated from Sobol sequences, paired with antithetic variates and the Brownian bridge construction[11].

Let N_{steps} denote the number of discrete time steps on the simulation grid, and let d denote the active factor dimension. The total dimensionality D of the initialised Sobol sequence per path is $D = N_{\text{steps}} \times d$. The generator draws low-discrepancy uniform samples $U \sim \mathcal{U}(0,1)^D$ across this high-dimensional space for only half of the required simulation paths, $N_{\text{paths}}/2$. These uniform draws are then mapped into the standard normal domain by applying the inverse cumulative distribution function of the standard normal distribution, $Z^{(i)} = \Phi^{-1}(U^{(i)})$.

Next, a Brownian bridge transformation assigns the lowest-discrepancy dimensions of the Sobol sequence to the coarsest features of the simulated paths. Following Glasserman [13], the bridge samples the terminal value first and then each intermediate point conditionally on its two nearest sampled neighbours, bisecting the grid one level at a time. The construction is linear in the standard normal variates Z, so the library computes it once as a matrix M that maps them to standardised increments and bridges all paths with the single product $Z_{\text{bridge}} = ZM^\top$. These bridged variables are then scaled by the square root of the discrete time step to produce the independent Brownian increments for the first half of the paths, $\Delta Z_{\text{half}} = Z_{\text{bridge}}\sqrt{\Delta t}$.

Finally, the complete set of independent Brownian increments is reconstructed by concatenating the generated paths with their antithetic mirrors,

$$\Delta Z = \begin{bmatrix} \Delta Z_{\text{half}} \\ -\Delta Z_{\text{half}} \end{bmatrix}. \tag{38}$$

4.3 Engines

4.3.1 Rough SABR Engine

The Rough SABR engine prices European swaptions under the model of Section 3.1 without simulating the terminal forward rate. It uses the conditional Monte Carlo of Section 3.1.1, which conditions on the path of the volatility driver and integrates the orthogonal shock $\Delta Z^\perp$ out in closed form, so only the increments ΔZ of the volatility driver are simulated. Because that volatility path is recovered from a single convolution of ΔZ against the Volterra kernel, each swaption price reduces to one kernel projection followed by a closed-form Bachelier evaluation.

Let $\Delta Z \in \mathbb{R}^{N_{\text{steps}} \times 1}$ be the vector of independent Brownian increments of Section 4.2. The fractional driver $\tilde{Z}$ of the volatility process is built from these increments by the hybrid construction of Equation 36, the product of the tail kernel matrix with the increments plus the exactly simulated near cell,

$$\tilde{Z} = K^{\text{tail}}\Delta Z + \frac{\xi_{\cdot,1}}{\Gamma(H+1/2)}. \tag{39}$$

The discrete unit-mean volatility path follows by exponentiating this driver against the variance compensator $\mathcal{V}(t_i)$,

$$V(t_i) = \exp\big(\nu\tilde{Z}(t_i) - \mathcal{V}(t_i)\big), \qquad V(t_0) = 1. \tag{40}$$

Each increment is scaled by the volatility from the previous step t_{i-1} rather than the current one, which fixes the volatility before the increment is drawn and preserves the non-anticipating property of the Itô integral. Over a terminal maturity $T = N_{\text{steps}}\Delta t$, two running sums summarise each trajectory, the volatility-weighted shock $I_V(T)$ accumulating V against the incre-

[11] The low-discrepancy sequence is that of Sobol [30], randomised by the scrambling of Owen [24]. Moskowitz and Caflisch [23] combine Sobol points with the Brownian bridge, which reduces the effective dimension of the integrand [6]. Glasserman [13] reviews the three techniques.

ments ΔZ and the integrated variance $I_{V^2}(T)$ accumulating V^2 against time,

$$I_V(T) = \sum_{i=1}^{N_{\text{steps}}} V_{t_{i-1}} \Delta Z_{t_i}, \qquad I_{V^2}(T) = \sum_{i=1}^{N_{\text{steps}}} V_{t_{i-1}}^2 \Delta t. \tag{41}$$

As cumulative sums, they allow a single set of paths to cover every target maturity, since the moments for any expiry follow from the corresponding grid index without re-simulation.

The calibrated volatility level $\alpha(T)$ stays constant within each maturity and anchors the at-the-money volatility of Equation 3, its argument indexing the maturity rather than calendar time. All stochastic variation is therefore carried by V, and through it by $I_V(T)$ and $I_{V^2}(T)$. Conditioning on the simulated fractional variance path makes the discrete terminal forward rate normal, with path-specific conditional mean and variance

$$\hat{F}_{cond} = F_0 + \alpha(T)\rho I_V(T), \qquad \hat{\sigma}^2_{cond} = \alpha(T)^2(1-\rho^2) I_{V^2}(T), \tag{42}$$

so the simulated payoff of a European swaption reduces to the Bachelier formula evaluated at $\hat{F}_{cond}$ and $\hat{\sigma}^2_{cond}$.

Let P_{cond} denote this conditional Bachelier price for a given path. The Monte Carlo estimator of $\mathbb{E}[P_{cond}]$ still carries the sampling variance of the volatility driver ΔZ. The engine pairs it with a control variate, a second set of paths with unit volatility, $V_{t_i} = 1$, driven by the same increments ΔZ. Its two running sums reduce to the sum of the increments and the elapsed time, so its conditional moments are the deterministic-volatility analogues of $\hat{F}_{cond}$ and $\hat{\sigma}^2_{cond}$,

$$\hat{F}_{cv_cond} = F_0 + \alpha(T)\rho \sum_{i=1}^{N_{\text{steps}}} \Delta Z_{t_i}, \qquad \hat{\sigma}^2_{cv_cond} = \alpha(T)^2(1-\rho^2)T. \tag{43}$$

The conditional control-variate payoff P_{cv_cond} is then obtained by evaluating the Bachelier formula at these simplified moments. Because the unconditional Bachelier price $P_{true} = \mathbb{E}^A[(F(T)-K)^+]$ under $F(T) \sim \mathcal{N}(F_0, \alpha(T)^2 T)$ is known in closed form, it serves as the analytical anchor of the estimator. Thus, the Conditional Monte Carlo price is computed as

$$P_{MC} = \bar{P}_{cond} - c_{\text{cv}} \left(\bar{P}_{cv_cond} - P_{true} \right), \tag{44}$$

where $\bar{P}_{cond}$ and $\bar{P}_{cv_cond}$ denote the sample means of the conditional SABR payoffs and the conditional Bachelier payoffs over the N_{paths} simulated trajectories, respectively. Finally, the control-variate coefficient c_{cv} is estimated on the simulated batch as the Ordinary Least Squares estimator [13], the sample covariance divided by the sample variance of the control variate,

$$c_{\text{cv}} = \frac{\sum_{m=1}^{N_{\text{paths}}} (P^{(m)}_{cond} - \bar{P}_{cond})(P^{(m)}_{cv_cond} - \bar{P}_{cv_cond})}{\sum_{m=1}^{N_{\text{paths}}} (P^{(m)}_{cv_cond} - \bar{P}_{cv_cond})^2}, \tag{45}$$

where the superscript (m) denotes the value on the m-th simulated path. Prices are converted to normal implied volatilities by Newton iteration on the Bachelier formula, with the Bachelier vega as the derivative.

The coefficient is estimated on the same paths that it corrects. This suits calibration, where the estimator enters an objective function and carries no error bar. Wherever a standard error is quoted, as in the out-of-sample repricing of `tests/oos.py`, c_{cv} is estimated instead on an independent pilot batch. The coefficient is then independent of the payoffs it corrects, which removes the bias that an in-batch coefficient introduces [13]. The library applies the control variate in both calibration stages.

4.3.2 Rough SABR FMM Engine

The FMM engine maps independent standard Brownian motions into the correlated drivers of the system through the lower-triangular Cholesky factor L of the calibrated correlation matrix Σ. Since Σ has unit diagonal, the rows of L have unit Euclidean norm, and the generated drivers keep unit variance and the target correlations.

Let $\Delta Z \in \mathbb{R}^{N_{\text{steps}} \times d}$ be the matrix of independent Brownian increments, with $d = N+1$. The correlated increments that drive the forward rates are obtained by projecting the raw increments through the Cholesky factor, $\Delta W = \Delta Z L^\top$, with the forward rates in columns 1 to N. Since L is lower-triangular, its first row is $(L_{00}, 0, \ldots, 0)$, and the volatility driver depends on the first independent factor alone. The engine therefore builds the fractional driver from that factor only, by the hybrid construction of Equation 39, and scales it by L_{00}, which equals one because Σ has unit diagonal,

$$\tilde{Z}_{\text{vol}} = L_{00} \left[K^{\text{tail}} \Delta Z_{\cdot,0} + \frac{\xi_{\cdot,1}}{\Gamma(H+1/2)} \right], \tag{46}$$

with the near-cell noise $\xi_{\cdot,1}$ drawn for this factor only. The remaining N factors enter the simulation only through ΔW.

With the fractional driver reconstructed, the engine evaluates the stochastic volatility paths. As in the Rough SABR engine of Section 4.3.1, each increment is scaled by the volatility of the previous step, so the volatility is predictable and the discrete Itô sum is non-anticipating. Using the global volatility of volatility ν and the discrete variance compensator of Equation 37, the volatility level $V_j(t_i)$ is evaluated as

$$V_j(t_i) = \exp\left(\nu \tilde{Z}_{\text{vol}}(t_i) - \mathcal{V}(t_i) \right), \quad V_j(t_0) = 1, \tag{47}$$

which is the same for every tenor j.

The final step of the simulation integrates the underlying forward rates. With $C(\cdot) = 1$, the volatility of the j-th forward rate is $\alpha_j V_j(t_i)$. The drift base $\mu_j(0)$ is the frozen drift of Equation 20, where each correlation $\Sigma_{j,k}$ is the dot product of the loading vectors of L, $\Sigma_{j,k} = \sum_{m=0}^{N} L_{j,m} L_{k,m}$.

Finally, the path integration is evaluated as a discretised summation,

$$F_j(t_n) = F_j(0) + \sum_{i=1}^{n} \mu_j(0) V_j(t_{i-1})^2 \Delta t + \sum_{i=1}^{n} \alpha_j V_j(t_{i-1}) \Delta W_j(t_i), \tag{48}$$

where $\Delta W_j(t_i)$ is the increment over $(t_{i-1}, t_i]$.

5 Calibration

Calibrating the model requires finding the parameter set that minimises the discrepancy between the model-generated swaption implied volatilities and those observed in the market. To keep the optimiser away from an over-parameterised search space, the process is divided into two sequential stages. The first stage determines the rough volatility parameters of the model of Section 3.1, the base volatility, the volatility of volatility and the spot-volatility correlations, on the 1Y tenor column of the swaption matrix. It runs at a fixed Hurst exponent, selected from a grid of candidate values. The second stage then fits the free angular coordinates of the $(N+1) \times (N+1)$ correlation matrix of Section 3.2.2, using the at-the-money co-terminal swaptions.

An objective computed by Monte Carlo calls the engine at every evaluation, and the optimiser evaluates it repeatedly, so a pure Monte Carlo calibration runs one simulation per evaluation. A calibration on the closed-form approximation of Section 3.1.2 alone needs no simulation, but

it carries the error of the approximation into the calibrated parameters, and the pricers that use them then misprice systematically.

To achieve Monte Carlo accuracy without surrendering the speed of analytical calibration, we adapt the Approximation and Model Management Optimization framework.

5.1 Approximation and Model Management Optimization Framework

The Approximation and Model Management Optimization framework of Alexandrov et al. [2] solves the subproblems of a trust-region method on a cheaper, lower-fidelity model, and calls the higher-fidelity model at every iteration to correct the surrogate and to test the step. When the corrected surrogate matches the value and the gradient of the high-fidelity objective at each iterate, the iterates converge to a stationary point of the high-fidelity problem from any starting point.

Let $x \in \mathbb{R}^n$ represent the vector of parameters being optimised, such as the marginal parameters or the correlation angles. We define the high-fidelity model, $f_{HF}(x)$, as the Rough SABR engine of Section 4.3.1, and the low-fidelity model, $f_{LF}(x)$, as the closed-form surrogate presented in Section 3.1.2. Although $f_{LF}(x)$ is cheap to evaluate and guides the search direction, it carries the asymptotic pricing errors of Section 3.1.2. AMMO corrects the surrogate at each iteration and substitutes it for the high-fidelity function inside the trust-region subproblem, keeping the cost of $f_{LF}(x)$ and the accuracy of $f_{HF}(x)$.

At any iteration k of the optimisation, the trust-region subproblem requires a local surrogate model $s_k(x)$ that matches the high-fidelity objective near the current anchor point x_k. To align the low-fidelity model $f_{LF}(x)$ with the high-fidelity model $f_{HF}(x)$, we apply the static multiplicative β-correction of Chang et al. [7][12].

Let $\beta(x_k)$ be a multiplicative correction factor evaluated at the current anchor point,

$$\beta(x_k) = \frac{f_{HF}(x_k)}{f_{LF}(x_k)}. \tag{49}$$

Scaling the low-fidelity model with this factor generates a basic surrogate evaluation for any proposed parameter set x,

$$s_k(x) = \beta(x_k) f_{LF}(x). \tag{50}$$

By construction $s_k(x_k) = f_{HF}(x_k)$, so the surrogate and the high-fidelity model coincide at the centre of the trust region, so zero-order consistency holds at the anchor point.

However, zero-order consistency alone does not give the convergence result of Alexandrov et al. [2], which requires the gradients to match as well. Standard nonlinear optimisation algorithms set the search direction from the local slope of the objective function. If the surrogate matches only the function value but has a different gradient than the underlying objective, the subproblem will guide the optimiser along an incorrect direction, leading to persistent step rejections. For the surrogate to provide a valid search direction, AMMO requires first-order consistency, that is, the gradient of the surrogate must match the gradient of the high-fidelity model at the anchor point, $\nabla s_k(x_k) = \nabla f_{HF}(x_k)$. To satisfy this condition, the static multiplicative correction is replaced with a dynamic, first-order approximation. We construct a first-order Taylor expansion of the correction term to predict how the ratio between the high-fidelity and low-fidelity models evolves as the optimiser explores the surrounding parameter space.

The first-order Taylor expansion of the correction factor around the current anchor point x_k, for a proposed point x, is

$$\beta_i(x) \approx \beta_i(x_k) + \nabla\beta_i(x_k)^\top (x - x_k), \tag{51}$$

where $\beta_i(x) = f_{HF,i}(x)/f_{LF,i}(x)$ for each target i, and $\nabla\beta_i(x_k)$ is the gradient of the i-th correction factor with respect to the optimisation parameters. To compute the gradient without

[12] Unrelated to the CEV exponent β of Section 3.2, which is zero throughout.

resorting to finite differences on the expensive correction ratio itself, we apply the quotient rule to $\beta_i(x)$. Differentiating the ratio of the high-fidelity and low-fidelity functions expresses the gradient in terms of their individual derivatives,

$$\nabla\beta_i(x_k) = \frac{f_{LF,i}(x_k)\nabla f_{HF,i}(x_k) - f_{HF,i}(x_k)\nabla f_{LF,i}(x_k)}{[f_{LF,i}(x_k)]^2}. \tag{52}$$

This formulation allows us to construct the Jacobian matrix of the correction factor, $J_{\beta(x_k)} \in \mathbb{R}^{m\times n}$, by assembling the individual gradients $\nabla\beta_i(x_k)^\top$ as its rows. The assembly requires only the base evaluations, and the Jacobians of the high-fidelity model (J_{HF}) and the low-fidelity model (J_{LF}) at the anchor point. Once computed at x_k, this fixed Jacobian matrix captures the local discrepancy between the gradients of the simulation and the surrogate, providing the directional adjustments necessary to achieve first-order consistency.

With the Jacobian of the correction factor computed at the anchor point x_k, we can construct the fully corrected surrogate model. For any trial point x explored by the optimisation algorithm, we first compute the updated correction vector using the first-order Taylor expansion of Equation 51,

$$\beta_{\text{trial}}(x) = \beta(x_k) + J_{\beta(x_k)}(x - x_k). \tag{53}$$

The final surrogate evaluation $s_k(x)$ is then obtained by scaling the low-fidelity evaluation $f_{LF}(x)$ by this approximated correction factor. Evaluated element-wise for each calibration target i, the corrected surrogate reads

$$s_{k,i}(x) = \beta_{\text{trial},i}(x) f_{LF,i}(x) = \Big[\beta_i(x_k) + \nabla\beta_i(x_k)^\top (x - x_k)\Big] f_{LF,i}(x). \tag{54}$$

This surrogate replaces the expensive high-fidelity model during the search for the next parameter iterate. With y_{mkt} the target market implied volatilities and E the merit of the stage defined below, the optimisation algorithm solves the bounded trust-region subproblem for the step vector

$$\min_{p_k} \; E\big(s_k(x_k + p_k)\big) \quad \text{subject to} \quad \|p_k\|_\infty \le \Delta_k, \tag{55}$$

where Δ_k is the current trust-region radius. Once the optimiser identifies the trial step p_k using the cheap surrogate, AMMO completes the iteration loop by evaluating the high-fidelity model $f_{HF}(x_k + p_k)$ at the proposed coordinates. To determine whether this step should be accepted or rejected, the algorithm computes the ratio ψ_k of the actual reduction achieved in the high-fidelity objective to the reduction predicted by the surrogate model,

$$\psi_k = \frac{E_{HF}(x_k) - E_{HF}(x_k + p_k)}{E_{HF}(x_k) - E_{s_k}(x_k + p_k)}, \tag{56}$$

where $E_{HF}(x) = E\big(f_{HF}(x)\big)$ and $E_{s_k}(x) = E\big(s_k(x)\big)$ apply the merit of the stage to the two models, the vega-weighted squared error of Section 5.2 in Stage 1 and the penalised objective J of Equation 59 in Stage 2. The zero-order consistency $s_k(x_k) = f_{HF}(x_k)$ established above gives $E_{s_k}(x_k) = E_{HF}(x_k)$, so the denominator coincides with the reduction predicted by the surrogate.

The step is accepted whenever it reduces the high-fidelity merit, $E_{HF}(x_k + p_k) < E_{HF}(x_k)$, in which case the anchor is updated to $x_{k+1} = x_k + p_k$; otherwise the step is rejected and the anchor is retained. The ratio ψ_k then sets the trust-region radius. When ψ_k is close to one the surrogate has predicted the descent and Δ_k is expanded, whereas a value near zero or negative signals that the low-fidelity approximation has broken down and Δ_k is contracted to force a smaller step.

[13]The gradient condition holds only approximately. In Stage 1 the library computes J_{LF} by forward differences and J_{HF} pathwise with c_{cv} held fixed, and in Stage 2 it computes J_{HF} and J_{LF} by automatic differentiation, J_{HF} on 2,048 paths against 32,768 for f_{HF}.

Algorithm 1 AMMO variable-fidelity trust-region calibration

Require: high-fidelity f_{HF}, surrogate f_{LF}, target y_{mkt}, start x_0, radius Δ_0, radius bounds $\Delta_{\min} < \Delta_{\max}$, tolerance ε_E, bounds $[\ell, u]$, budget K

Const. $r_1 = 0.1,\ r_2 = 0.75,\ c_1 = 0.5,\ c_2 = 2.0$; merit E, the calibration objective of the stage

for $k = 0, 1, \ldots, K-1$ **do**

  evaluate $f_{\mathrm{HF}}(x_k)$, $J_{\mathrm{HF}}(x_k)$ and $f_{\mathrm{LF}}(x_k)$, $J_{\mathrm{LF}}(x_k)$ ▷ HF from the previous iteration

  $\beta_k \leftarrow f_{\mathrm{HF}}(x_k) \oslash f_{\mathrm{LF}}(x_k)$

  $J_{\beta_k} \leftarrow \mathrm{diag}\big(f_{\mathrm{LF}}(x_k)\big)^{-1} J_{\mathrm{HF}}(x_k) - \mathrm{diag}\big(f_{\mathrm{HF}}(x_k) \oslash f_{\mathrm{LF}}(x_k)^2\big) J_{\mathrm{LF}}(x_k)$

  $s_k(x) \leftarrow \big(\beta_k + J_{\beta_k}(x - x_k)\big) \odot f_{\mathrm{LF}}(x)$ ▷ $s_k(x_k) = f_{\mathrm{HF}}(x_k),\ \nabla s_k(x_k) = \nabla f_{\mathrm{HF}}(x_k)$[13]

  $p_k \leftarrow \arg\min_p E\big(s_k(x_k + p)\big)$ s.t. $\|p\|_\infty \le \Delta_k,\ x_k + p \in [\ell, u]$

  evaluate $f_{\mathrm{HF}}(x_k + p_k)$ ▷ the true HF error at the trial step

  $$\psi_k \leftarrow \frac{E\big(f_{\mathrm{HF}}(x_k)\big) - E\big(f_{\mathrm{HF}}(x_k + p_k)\big)}{E\big(f_{\mathrm{HF}}(x_k)\big) - E\big(s_k(x_k + p_k)\big)}$$

  if $E\big(f_{\mathrm{HF}}(x_k + p_k)\big) < E\big(f_{\mathrm{HF}}(x_k)\big)$ **then** $x_{k+1} \leftarrow x_k + p_k$

  else $x_{k+1} \leftarrow x_k$ ▷ accept iff HF merit decreases

  $$\Delta_{k+1} \leftarrow \begin{cases} c_1 \|p_k\|_\infty & \psi_k < r_1 \\ \min(c_2\, \Delta_k, \Delta_{\max}) & \psi_k > r_2 \\ \Delta_k & \text{otherwise} \end{cases}$$

  if $\Delta_{k+1} \le \Delta_{\min}$ **or** the reduction of the HF merit falls below ε_E **or** two consecutive rejections **then break**

end for

return x_{k+1}

5.2 Stage 1: Rough Volatility Parameters

To calibrate the Stage-1 parameters $\Theta = \{\nu, \alpha_{1,\ldots,N_{\mathrm{exp}}}, \rho_{1,\ldots,N_{\mathrm{exp}}}\}$ across the swaption surface, we formulate the calibration as a weighted non-linear least-squares inverse problem. Rather than weighting all implied-volatility errors equally, we follow West [31], who weights the squared volatility error of each option by its vega, and use the Bachelier vega of each swaption.

Applying Bachelier vega across a full surface, however, causes the natural $\sqrt{T}$ scaling of standard vega to favour long-dated maturities over short-dated ones. To resolve this, we normalise the vega of each swaption by the at-the-money vega of the same expiry, with objective function

$$\hat{\Theta} = \arg\min_{\Theta} \sum_{i=1}^{N_{\mathrm{opts}}} \mathrm{wtg}_i \left[\sigma_i^{\mathrm{mod}}(\Theta) - \sigma_i^{\mathrm{mkt}}\right]^2, \tag{57}$$

where the static weight wtg_i for each swaption i is defined by the normal d-term of the Bachelier formula, bounded from below by a liquidity floor,

$$\mathrm{wtg}_i = \max\left[\exp\left(-\frac{1}{2} d_i^2\right), \mathrm{wtg}_{\mathrm{floor}}\right], \quad \text{where} \quad d_i = \frac{k_i}{\sigma_i^{\mathrm{mkt}} \sqrt{T_i}}, \tag{58}$$

and k_i is the absolute strike offset from the forward rate, as in Section 3.1.2. The exponential term is the ratio of the Bachelier vega at the market volatility to the at-the-money vega of the same expiry, so the weight depends on the standardised moneyness d_i alone and equals one at the money. In our implementation, we set the liquidity floor to $\mathrm{wtg}_{\mathrm{floor}} = 0.25$. This focuses the optimiser on the ATM spine ($\mathrm{wtg}_i \approx 1.0$) while maintaining a 25% minimum penalty weighting in

the wings to preserve the shape of the smile. The parameter space of the trust-region optimiser is further bounded, with $\nu \in [0.001, 10.0]$, $\alpha \in [0.0001, 1.0]$, and $\rho \in [-0.999, 0.999]$. Stage 1 stops when an accepted step lowers the vega-weighted RMSE by less than 0.01 bps, when the surrogate predicts a reduction below 0.001 bps, or when the radius falls to 0.001. These tolerances are fixed in the code rather than in the configuration.

5.3 Stage 2: Correlation Matrix

Once the marginals are calibrated, the second stage fits the angles $\omega_{i,j}$ of the $(N+1) \times (N+1)$ correlation matrix of Section 3.2.2 to the at-the-money co-terminal swaptions. Pricing them through the joint dynamics would require simulating the whole forward curve, so both AMMO models price instead the mapped process of Section 3.3. On the same mapped parameters, the high-fidelity model runs the Rough SABR engine of Section 4.3.1, and the low-fidelity model the closed-form approximation of Section 3.1.2.

Before the optimisation, the matrix is anchored, to keep the structure inherited from Stage 1 and to remove directions that the quotes cannot identify. Following Adachi et al. [1], the first column is set to the marginal spot-volatility correlations, $\omega_{i,0} = \arccos \rho_i$, so that the correlation of each forward rate with the volatility process stays at its Stage 1 value. At the short end, $\omega_{2,1} = 0$ and $\omega_{i,2} = \pi/2$ for $i \geq 3$. The first anchor gives the first two forward rates the largest correlation that their spot-volatility correlations allow, $\mathrm{corr}(F_1, F_2) = \cos(\omega_{1,0} - \omega_{2,0})$. The second fixes angles that, after the first anchor, do not change Σ, so no quote can identify them[14].

The remaining free angular coordinates of all rows are assembled into a single parameter vector and calibrated jointly. We minimise the regularised objective over the whole matrix in a single AMMO trust-region loop, matching the free angles of each row to the co-terminal swaptions at its target maturity. The surrogate supplies both the objective and its gradient at the high-fidelity anchor.

5.3.1 Regularisation

alibrating Σ to swaption volatilities is an ill-posed inverse problem. In fact, adjacent forward rates are highly correlated, so many different sets of angles reprice the co-terminal swaptions almost equally well, and the quotes alone do not determine Σ. The RBM parameterisation guarantees that every candidate is a valid correlation matrix, but it does not choose among the near-equivalent fits. We choose by adding two penalties to the calibration objective.

Crépey [8] calibrates a local volatility by adding to the squared price error a Tikhonov penalty that keeps the solution close to a prior, and smooth. We follow this framework with two penalties on the rows of Σ, one on their curvature and one on their distance to a target matrix,

$$J(\theta) = \sum_k \left[\sigma_{\mathrm{mod}}(k, \theta) - \sigma_{\mathrm{mkt}}(k)\right]^2 + \lambda\, \zeta \sum_i \left[R_h(\Sigma_i) + R_v(\Sigma_i)\right], \tag{59}$$

where the data term sums unweighted squared errors over the at-the-money quotes, θ collects the free angles, Σ_i is the i-th row of Σ, and ζ is a scaling constant defined below. The horizontal term penalises the curvature of each row through its second differences,

$$R_h(\Sigma_i) = \sum_j \left(\Sigma_{i,j+1} - 2\Sigma_{i,j} + \Sigma_{i,j-1}\right)^2, \tag{60}$$

[14] With $\omega_{2,1} = 0$, row 2 of B is $(\cos\omega_{2,0}, \sin\omega_{2,0}, 0, \ldots, 0)$, so rows 0 to 2 are zero from column 2 onwards. A rotation of these columns moves only the rows $i \geq 3$ and leaves every product of two rows of B, and so Σ, unchanged. Such a rotation can always clear column 2 in these rows, which sets $\omega_{i,2} = \pi/2$, so fixing these angles removes no correlation matrix.

and the vertical term pulls each row towards a target correlation matrix Σ^{tgt},

$$R_v(\Sigma_i) = \sum_j \left(\Sigma_{i,j} - \Sigma^{\mathrm{tgt}}_{i,j}\right)^2. \tag{61}$$

Both terms use only the entries with $3 \le j \le i-1$, to the left of the diagonal. The correlations with the volatility driver are excluded because Stage 1 fixes them. Those with the first two forward rates are excluded because the anchor $\omega_{2,1} = 0$ of the previous subsection gives these two rates a partial correlation of one given the volatility driver, which a target decaying with tenor separation cannot reproduce.

The constant ζ is the ratio of the squared data residual to the penalty at the starting point θ_0 of the optimisation (Appendix A). At $\lambda = 1$ the two terms of J are equal at θ_0, so λ measures the weight of the penalty relative to the fit.

The target should supply what the swaption quotes leave undetermined, the decorrelation of the forward rates with tenor separation. We estimate it from the history of the forward curve. The sample correlation $\hat{\Sigma}$ of one year of weekly changes of the forward rates, constructed in Appendix A, is noisy, so we summarise it by the two-parameter exponentially decreasing parameterisation of Brigo and Mercurio [5],

$$\Xi_{i,j}(\varsigma, \rho_\infty) = \rho_\infty + (1 - \rho_\infty)\, e^{-\varsigma|i-j|}, \tag{62}$$

with decay rate $\varsigma \ge 0$ per year of tenor separation, since $|i-j|$ counts years on the annual grid, and long-end level $\rho_\infty \in [0, 1)$.

Equation 62 describes how the forward rates are correlated with one another, but in the model each forward rate is also correlated with the volatility driver, through the first column of Σ that Stage 1 has fixed. In the factor B of Equation 22, every entry of row i after the first carries the factor $\sin\omega_{i,0} = \sqrt{1-\rho_i^2}$, so

$$\Sigma_{i,j} = \rho_i\rho_j + \sqrt{(1-\rho_i^2)(1-\rho_j^2)}\;\Sigma^{\perp}_{i,j}, \tag{63}$$

where $\Sigma^{\perp}_{i,j}$ is the partial correlation of the two rates given the volatility driver, set by the angles $\omega_{i,1}, \omega_{i,2}, \ldots$ that Stage 2 calibrates. The first term is fixed by Stage 1, so Stage 2 acts on Σ only through the correlation matrix $\Sigma^{\perp}$, and a target imposed on Σ directly is in general out of reach of every angle configuration (Appendix A). Thus, we apply Equation 62 to the part that Stage 2 controls, substituting $\Sigma^{\perp} = \Xi$ in Equation 63,

$$\Sigma^{\mathrm{tgt}}_{i,j} = \rho_i\rho_j + \sqrt{(1-\rho_i^2)(1-\rho_j^2)}\;\Xi_{i,j}(\varsigma, \rho_\infty). \tag{64}$$

We define Σ^{tgt} as the correlation matrix that the model would have if its partial correlations decayed as in Equation 62. It has unit diagonal and, together with the first column, is positive semi-definite for every admissible ς and ρ_∞, since Ξ is a convex combination of the matrix of ones and the correlation matrix $e^{-\varsigma|i-j|}$ of a first-order autoregressive process. We fit the two parameters to $\hat{\Sigma}$ through Equation 64, by unweighted least squares on the entries that the penalty uses. The target depends only on the history of the curve and on the Stage-1 correlations, not on the quotes that Stage 2 fits.

With the target fixed, only the weight λ is left to define. A small λ lets the angles follow the quotes, including their noise, while a large λ pulls Σ towards Σ^{tgt} at the cost of the fit. Since the target carries the decorrelation that the quotes do not determine, we want the largest λ that the quotes do not contradict, judged on quotes that the calibration has not seen. We choose λ by cross-validation with the one-standard-error rule described by Hastie et al. [16]. The at-the-money quotes are split into five groups within each row of Σ, each group is left out of the data term in turn and repriced at the angles fitted to the others, and the split is drawn twice. Within the list of fixed candidates Λ, the rule takes the largest λ whose cross-validation error lies within one standard error of the smallest. Appendix A gives estimator and standard error.

6 Valuation

The library includes the pricers for three exotic instruments, constant-maturity caplets and floorlets, constant-maturity spread options, and Bermudan swaptions. The trade economics, market conventions, and standard analytical treatments of these products follow Brigo and Mercurio [5]. All three pricers share a single valuation convention, which is stated in Section 6.1 and then applied to each instrument in turn.

6.1 Common Valuation Convention

Let $\mathcal{T} = \{T_0 < T_1 < \cdots < T_N\}$ be the tenor structure of Section 3.2, with year fractions $\tau_i = T_i - T_{i-1}$ and backward-looking forwards F_i. All expectations are taken under the terminal measure $\mathbb{Q}^{T_N}$ associated with the numéraire $P(\cdot, T_N)$, which is the measure in which the forward curve is simulated.

Consider an $\mathcal{F}_{T_e}$-measurable, integrable payoff $X(T_e)$ settling at a date $T_e \in \mathcal{T}$. Writing $D(t,T)$ for the stochastic discount factor, a change of numéraire [11] from the bank account to $P(\cdot, T_N)$ gives the time-zero value

$$\pi_0 = \mathbb{E}\big[D(0,T_e)\,X(T_e)\big] = P(0,T_N)\,\mathbb{E}^{T_N}\left[\frac{X(T_e)}{P(T_e,T_N)}\right], \tag{65}$$

where the second equality uses $P(T_e,T_e) = 1$.

Every pricer in this section is an instance of Equation 65, and we refer to the quantity $1/P(T_e,T_N)$ as the deflator. Swaption values are conventionally quoted under the annuity measure, and the two conventions are related by a further change of numéraire. For a payoff settling at T_e and an annuity $A_{a,b}$,

$$P(0,T_N)\,\mathbb{E}^{T_N}\left[\frac{X(T_e)}{P(T_e,T_N)}\right] = A_{a,b}(0)\,\mathbb{E}^{a,b}\left[\frac{X(T_e)}{A_{a,b}(T_e)}\right], \tag{66}$$

where $A_{a,b}(0) = \sum_{i=a+1}^{b} \tau_i P(0,T_i)$ is the time-zero annuity.

At the settlement date, the path-wise bonds are reconstructed from the simulated forwards through

$$P(T_e,T_k) \;=\; \prod_{i=e+1}^{k} \big(1+\tau_i F_i(T_e)\big)^{-1}, \qquad e \le k \le N, \tag{67}$$

so the deflator, the annuity $A_{a,b}(T_e)$, and the forward swap rate $S_{a,b}(T_e)$ of Section 3.3 are all functions of the single curve $\{F_i(T_e)\}_{i>e}$. For payoffs with a single fixing, the simulation is carried only as far as that fixing.

The discounting and the payoff are simulated jointly. The deflator and the payoff are measurable with respect to the same curve and are not independent.

Sensitivities are computed by automatic differentiation. Every operation between the simulated increments and the deflated payoff is recorded on the automatic-differentiation graph, so a single adjoint pass returns the sensitivities of the price to the initial curve and to the calibrated volatility parameters [12].

The library supports only the $\beta = 0$ (Bachelier) specification, under which the payoff is continuous in the simulated rates and differentiable except at the strike, which a simulated rate reaches with probability zero, and the pathwise Greeks are unbiased. [13].

6.2 Constant-Maturity Coupons

A constant-maturity caplet pays the positive part of the excess of the c-year par swap rate over a strike K, accrued by the year fraction δ, on the fixing date T_a. With $b = a + c$ and the payoff

settling at the fixing date, Equation 65 gives

$$\pi_{\text{cms}} = P(0,T_N)\,\delta\,\mathbb{E}^{T_N}\left[\frac{\big(S_{a,b}(T_a)-K\big)^+}{P(T_a,T_N)}\right], \tag{68}$$

and the floorlet follows on replacing the payoff by $\delta\big(K - S_{a,b}(T_a)\big)^+$. The coupon is taken to settle at its fixing date T_a. Settlement at a later date $T_p > T_a$ would introduce the additional path-wise factor $P(T_a,T_p)$ inside the expectation and is not treated here.

No convexity adjustment is required[15]. The adjusted rate is recovered directly as the deflator-weighted rate

$$S^{\text{cms}}_{a,b} \;=\; \mathbb{E}^{T_a}\big[S_{a,b}(T_a)\big] \;=\; \frac{\mathbb{E}^{T_N}\big[S_{a,b}(T_a)/P(T_a,T_N)\big]}{\mathbb{E}^{T_N}\big[1/P(T_a,T_N)\big]}, \tag{69}$$

which is a ratio of two Monte Carlo averages over the same paths.

6.3 Constant-Maturity Spread Options

A spread option is written on two par swap rates of different underlying tenors sharing a single fixing date. With $b_1 = a+c_1$, $b_2 = a+c_2$ and $c_1 > c_2$, the payoff at T_a is $\delta\big(S_{a,b_1}(T_a) - S_{a,b_2}(T_a) - K\big)^+$ and its value is

$$\pi_{\text{spread}} = P(0,T_N)\,\delta\,\mathbb{E}^{T_N}\left[\frac{\big(S_{a,b_1}(T_a)-S_{a,b_2}(T_a)-K\big)^+}{P(T_a,T_N)}\right], \tag{70}$$

which coincides with the T_a-forward-measure expression $P(0,T_a)\,\mathbb{E}^a[\,\cdot\,]$ by a further change of numéraire. Both legs are tenor slices of the same fixing-date curve $\{F_i(T_a)\}_{i>a}$, so a single application of Equation 67 over the longer window reconstructs both. Their joint law, and in particular their cross-correlation, is inherited from the calibrated matrix Σ and is not supplied exogenously.

6.4 Bermudan Swaptions

A payer Bermudan swaption gives the holder the right to enter, at any date T_e of an exercise set $\mathcal{E} = \{T_k,\dots,T_h\} \subset \mathcal{T}$, the co-terminal payer swap running from T_e to a fixed terminal date T_n at the fixed rate K. The deflated exercise value is

$$Y(T_e) \;=\; \frac{A_{e,n}(T_e)}{P(T_e,T_N)}\,\big(S_{e,n}(T_e)-K\big)^+, \qquad T_e \in \mathcal{E}, \tag{71}$$

and, applying Equation 65 at the exercise date, the option value is the optimal-stopping problem

$$\pi_{\text{berm}} \;=\; P(0,T_N)\,\sup_{\vartheta}\;\mathbb{E}^{T_N}\big[Y(\vartheta)\big], \tag{72}$$

where the supremum runs over $\mathcal{E}$-valued stopping times. The value satisfies the backward recursion

$$U(T_h) = Y(T_h), \qquad U(T_e) = \max\Big\{Y(T_e),\;\mathbb{E}^{T_N}\big[U(T_{e+1}) \mid \mathcal{F}_{T_e}\big]\Big\}, \tag{73}$$

and since $Y(T_e)$ is already expressed in units of the terminal bond, no discount factor appears between exercise dates.

[15] The classical adjustment corrects the mismatch between the natural payment measure of the coupon and the swap measure under which the rate is a martingale [5, 14], by approximating the annuity with a flat yield curve at the swap rate and expanding to second order. Here the deflator is stochastic, correlated with the swap rate through the simulated curve, and retained inside the expectation, so the simulation produces the effect that the adjustment approximates.

We solve Equation 73 by least-squares Monte Carlo [20][16]. Working backward from T_h, the conditional expectation at each exercise date is estimated by regressing the realised continuation values on the basis $\{1, x, x^2\}$, where $x = S_{e,n}(T_e)/K$. The regression uses in-the-money paths only. If a date has fewer in-the-money paths than a fixed minimum, no continuation value is fitted and the option is not exercised.

The exercise policy is estimated on an independent pilot simulation, and the regression coefficients are then applied, unchanged, to a larger main simulation. The policy used for valuation is fixed before the main paths are drawn, and the estimator returns the value of an admissible but sub-optimal stopping rule, which is biased low relative to Equation 72 [13]. [17]

The regressions are excluded from the automatic-differentiation graph, while the exercised cashflows remain on it, so no gradient flows through the estimated exercise boundary [19, 26].

7 Limitations

The pricers support only the normal specification with a single global volatility of volatility.

The Sobol generator admits at most 21,201 dimensions, so the Bermudan valuation runs on a seeded antithetic pseudo-random driver without the low-discrepancy property of the driver of the European pricers.

[16] The Bermudan simulation needs more than the 21,201 dimensions that PyTorch's `SobolEngine`, with the direction numbers of Joe and Kuo [18], admits. It runs on a seeded pseudo-random driver, which keeps the antithetic pairing of Section 4.2 but not its low-discrepancy property.

[17] In alterative, as shown by Glasserman [13], a single-pass regression, in which the same paths select the policy and receive its value, mixes this low bias with a high bias from foresight.

A About the Regularisation of the Correlation Matrix

This appendix details the Stage-2 penalty of Section 5.3, the scaling constant ζ, the target Σ^{tgt} and the selection of λ.

Constant ζ. The constant ζ of Equation 59 is evaluated at the starting point θ_0 of the optimisation, every free angle at 0.8 radians,

$$\zeta = \frac{\sum_k \left[\sigma_{\mathrm{mod}}(k,\theta_0) - \sigma_{\mathrm{mkt}}(k)\right]^2}{\sum_i \left[R_h(\Sigma_i(\theta_0)) + R_v(\Sigma_i(\theta_0))\right]}, \tag{74}$$

on the quotes of each calibration, including those of the cross-validation below.

Target Σ^{tgt}. Section 5.3 builds the target on the split of Equation 63, which we derive here from the parameterisation of Rapisarda et al. [27]. By Equation 23, every entry of row i of B after the first carries the factor $\sin\omega_{i,0}$, which equals $\sqrt{1-\rho_i^2}$ since $\cos\omega_{i,0} = \rho_i$ and $\omega_{i,0} \in [0,\pi]$. The row then reads

$$b_i = \big(\rho_i,\ \sqrt{1-\rho_i^2}\ u_i\big), \qquad u_i = \big(\cos\omega_{i,1},\ \sin\omega_{i,1}\cos\omega_{i,2},\ \ldots\big),$$

where u_i depends only on the angles that Stage 2 calibrates and is a unit vector [18]. The products $\Sigma_{i,j} = b_i^\top b_j$ then give Equation 63 with $\Sigma^\perp_{i,j} = u_i^\top u_j$. The meaning of $\Sigma^\perp$ is clearer in terms of the drivers. The factor B builds them from independent Brownian motions $\varepsilon_0,\ldots,\varepsilon_N$, with $(dZ, dW_1,\ldots,dW_N)^\top = B\,d\varepsilon$ and $Z = \varepsilon_0$, so row i splits each rate driver into a part along the volatility driver and a part orthogonal to it, as in the conditional Monte Carlo of Section 3.1.1, $dW_i(t) = \rho_i\,dZ(t) + \sqrt{1-\rho_i^2}\,dZ_i^\perp(t)$. Here $Z_i^\perp = u_i^\top(\varepsilon_1,\ldots,\varepsilon_N)^\top$ is a standard Brownian motion independent of Z. The correlation of $Z_i^\perp$ and $Z_j^\perp$ is $\Sigma^\perp_{i,j}$, the partial correlation of the two rates given the volatility driver.[19]

The target is fitted to the sample correlation $\hat\Sigma$ of the weekly changes $\Delta F_i(t_k) = F_i(t_k) - F_i(t_{k-1})$, over the year that ends on the calibration date, of the forward rates $F_i(t)$ of the curve of date t from $t+i-1$ to $t+i$ years, built as the initial curve of the model. We take absolute changes because the model is normal, and weekly ones, every fifth business day, because daily changes carry noise at the pillars of the curve. Dates on which a single forward rate jumps on the daily curve and reverts the next day are dropped as quote errors. The two parameters are fitted through Σ^{tgt} rather than through Ξ, since $\hat\Sigma$ estimates the full correlations of the forward rates, which include the part through the volatility driver that Σ^{tgt} carries and Ξ leaves out. On the entries that the penalty uses, they minimise $\sum_{3\le j<i\le N}\left[\Sigma^{\mathrm{tgt}}_{i,j}(\varsigma,\rho_\infty) - \hat\Sigma_{i,j}\right]^2$.

Weight λ. We select λ by the K-fold cross-validation of Hastie et al. [16], who fit the model to all groups of the data but one and compute the prediction error on the group left out, for each group in turn. We depart from their procedure in four places. First, the n at-the-money quotes are split into n_{g} groups within each row of Σ, where row i holds the quotes whose swap ends at T_i, the first to involve F_i and those to which the calibration matches the angles of row i. The quotes of a row are shuffled and dealt to the groups in turn from a random start, so every row keeps a quote outside each group and its angles stay identified. With $\hat\theta_{-q}(\lambda)$ minimising J of Equation 59 with group q left out of the data term, each quote k of group q is repriced with

[18] b_i is one and $1 = \|b_i\|^2 = \rho_i^2 + (1-\rho_i^2)\|u_i\|^2$.

[19] The split bounds what Stage 2 can reach. Since $\Sigma^\perp_{i,j} \in [-1,1]$, the angles reach only the values of $\Sigma_{i,j}$ in the interval $\rho_i\rho_j \pm \sqrt{(1-\rho_i^2)(1-\rho_j^2)}$, whose upper end $\cos(\omega_{i,0}-\omega_{j,0})$ is below one when $\rho_i \neq \rho_j$. A target on Σ itself, with $\Xi_{i,j}$ close to one, can then lie above the interval, whereas Ξ placed on $\Sigma^\perp$ has entries in $[0,1]$ and keeps every entry of Σ^{tgt} reachable.

the high-fidelity model at $\hat{\theta}_{-q}(\lambda)$, which gives its prediction error $e_k(\lambda)$, model minus market. Second, the split is drawn n_{d} times, to reduce the dependence on a single split, and the error is the root mean square of all $n_{\mathrm{d}}\, n$ prediction errors,

$$\mathrm{CV}(\lambda) = \Big(\frac{1}{n_{\mathrm{d}}\, n} \sum_{r=1}^{n_{\mathrm{d}}} \sum_{k=1}^{n} e_k^{(r)}(\lambda)^2 \Big)^{1/2}. \tag{75}$$

Third, Hastie et al. [16] take the standard error from the spread of the group errors, and we carry it to the root-mean-square scale. With $s(\lambda)$ the sample standard deviation of the $n_{\mathrm{g}}\, n_{\mathrm{d}}$ mean squared errors over the quotes left out, each an estimate of $\mathrm{CV}(\lambda)^2$, the delta method gives

$$\mathrm{SE}(\lambda) = \frac{s(\lambda)}{2\, \mathrm{CV}(\lambda) \sqrt{n_{\mathrm{g}}\, n_{\mathrm{d}}}}, \tag{76}$$

an approximation, since the calibrations of one draw share most of their quotes and the two draws reuse the same quotes. The one-standard-error rule then takes the most regularised candidate within $\mathrm{SE}(\lambda^*)$ of the smallest error $\mathrm{CV}(\lambda^*)$,

$$\hat{\lambda} = \max \big\{ \lambda \in \Lambda : \mathrm{CV}(\lambda) \le \mathrm{CV}(\lambda^*) + \mathrm{SE}(\lambda^*) \big\}. \tag{77}$$

The library uses $n_{\mathrm{g}} = 5$ groups and $n_{\mathrm{d}} = 2$ draws, and the grid $\Lambda = \{0.1, 0.316, 1, 3.16, 10, 31.6\}$, with a floor of 10^{-6} that stands for $\lambda \to 0$. The candidates are spaced by a factor of $\sqrt{10}$, the precision to which the rule determines $\hat{\lambda}$. Finally, the library scores the candidates in increasing order and stops at the first whose error exceeds the smallest so far by more than its standard error, returning the candidate before it. This gives $\hat{\lambda}$ of Equation 77 whenever the error keeps rising beyond it.

Disclaimer

Anthropic's Large Language Models assisted with writing, refactoring and reviewing the code, with drafting the text, and in writing the LaTeX commands. The literature review, the design of the library, the choice of experiments and the interpretation of the results are the author's. Every citation was checked against the cited work, and the author takes responsibility for the text and the code.

References

[1] Reo Adachi, Masaaki Fukasawa, Naoki Iida, Mitsumasa Ikeda, Yo Nakatsu, Ryota Tsurumi, and Tomohisa Yamakami. Rough SABR forward market model. Working paper, 2025.

[2] Natalia M. Alexandrov, Robert Michael Lewis, Clyde R. Gumbert, Lawrence L. Green, and Perry A. Newman. Approximation and model management in aerodynamic optimization with variable-fidelity models. *Journal of Aircraft*, 38(6):1093–1101, 2001.

[3] Christian Bayer, Peter Friz, and Jim Gatheral. Pricing under rough volatility. *Quantitative Finance*, 16(6):887–904, 2016.

[4] Mikkel Bennedsen, Asger Lunde, and Mikko S. Pakkanen. Hybrid scheme for Brownian semistationary processes. *Finance and Stochastics*, 21(4):931–965, 2017.

[5] Damiano Brigo and Fabio Mercurio. *Interest Rate Models — Theory and Practice: With Smile, Inflation and Credit*. Springer Berlin Heidelberg, 2 edition, 2006. doi: 10.1007/978-3-540-34604-3.

[6] Russel E. Caflisch, William Morokoff, and Art B. Owen. Valuation of mortgage-backed securities using Brownian bridges to reduce effective dimension. *Journal of Computational Finance*, 1(1):27–46, 1997. doi: 10.21314/JCF.1997.005.

[7] K. J. Chang, R. T. Haftka, G. L. Giles, and P.-J. Kao. Sensitivity-based scaling for approximating structural response. *Journal of Aircraft*, 30(2):283–288, 1993.

[8] Stéphane Crépey. Calibration of the local volatility in a generalized Black–Scholes model using Tikhonov regularization. *SIAM Journal on Mathematical Analysis*, 34(5):1183–1206, 2003.

[9] Masaaki Fukasawa and Jim Gatheral. A rough SABR formula. *Frontiers of Mathematical Finance*, 1(1):81–97, 2022. doi: 10.3934/fmf.2021003.

[10] Paul Gassiat. Weak error rates of numerical schemes for rough volatility. *SIAM Journal on Financial Mathematics*, 2023. arXiv:2203.09298.

[11] Hélyette Geman, Nicole El Karoui, and Jean-Charles Rochet. Changes of numéraire, changes of probability measure and option pricing. *Journal of Applied Probability*, 32(2):443–458, 1995. doi: 10.2307/3215299.

[12] Michael B. Giles and Paul Glasserman. Smoking adjoints: fast Monte Carlo Greeks. *Risk*, 19(1):88–92, 2006.

[13] Paul Glasserman. *Monte Carlo Methods in Financial Engineering*, volume 53 of *Stochastic Modelling and Applied Probability*. Springer New York, 2003. doi: 10.1007/978-0-387-21617-1.

[14] Patrick S. Hagan. Convexity conundrums: Pricing CMS swaps, caps, and floors. *Wilmott Magazine*, 2003(2):38–45, 2003. doi: 10.1002/wilm.42820030211.

[15] Patrick S. Hagan, Deep Kumar, Andrew S. Lesniewski, and Diana E. Woodward. Managing smile risk. *The Best of Wilmott*, 1:249–296, 2002.

[16] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2 edition, 2009.

[17] John C. Hull and Alan White. Forward rate volatilities, swap rate volatilities, and the implementation of the LIBOR market model. *The Journal of Fixed Income*, 10(2):46–62, 2000. doi: 10.3905/jfi.2000.319268.

[18] Stephen Joe and Frances Y. Kuo. Constructing Sobol sequences with better two-dimensional projections. *SIAM Journal on Scientific Computing*, 30(5):2635–2654, 2008. doi: 10.1137/070709359.

[19] Matthias Leclerc, Qian Liang, and Ingo Schneider. Fast Monte Carlo Bermudan Greeks. *Risk*, 22(7):84–88, 2009.

[20] Francis A. Longstaff and Eduardo S. Schwartz. Valuing American options by simulation: A simple least-squares approach. *The Review of Financial Studies*, 14(1):113–147, 2001.

[21] Andrei Lyashenko and Fabio Mercurio. Looking forward to backward-looking rates: A modeling framework for term rates replacing LIBOR. *Risk*, 2019. Available at SSRN: https://ssrn.com/abstract=3330240.

[22] Ryan McCrickerd and Mikko S. Pakkanen. Turbocharging Monte Carlo pricing for the rough Bergomi model. *Quantitative Finance*, 18(11):1877–1886, 2018.

[23] Bradley Moskowitz and Russel E. Caflisch. Smoothness and dimension reduction in quasi-Monte Carlo methods. *Mathematical and Computer Modelling*, 23(8–9):37–54, 1996. doi: 10.1016/0895-7177(96)00038-6.

[24] Art B. Owen. Scrambling Sobol' and Niederreiter–Xing points. *Journal of Complexity*, 14 (4):466–489, 1998. doi: 10.1006/jcom.1998.0487.

[25] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035. Curran Associates, Inc., 2019.

[26] Vladimir V. Piterbarg. Computing deltas of callable LIBOR exotics in forward LIBOR models. *Journal of Computational Finance*, 7(3):107–144, 2004. doi: 10.21314/JCF.2004.109.

[27] Francesco Rapisarda, Damiano Brigo, and Fabio Mercurio. Parameterizing correlations: a geometric interpretation. *IMA Journal of Management Mathematics*, 18(1):55–77, 2007.

[28] Riccardo Rebonato. *Volatility and Correlation: The Perfect Hedger and the Fox*. John Wiley & Sons, 2nd edition, 2004. ISBN 978-0-470-09139-5.

[29] Marc Romano and Nizar Touzi. Contingent claims and market completeness in a stochastic volatility model. *Mathematical Finance*, 7(4):399–412, 1997. doi: 10.1111/1467-9965.00038.

[30] Ilya M. Sobol. On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics*, 7(4):86–112, 1967. doi: 10.1016/0041-5553(67)90144-9.

[31] Graeme West. Calibration of the SABR model in illiquid markets. *Applied Mathematical Finance*, 12(4):371–385, 2005.